

目 录

1 基于 face-api.js 的浏览器端人脸分析实验	1
1.1 实验定位与章节边界	1
1.2 项目结构与资源组织	2
1.2.1 静态项目结构	2
1.2.2 模型权重与本地加载	2
1.2.3 页面布局与显示容器	4
1.3 脚本加载与初始化流程	4
1.3.1 defer 脚本顺序	4
1.3.2 模型加载的并行等待	5
1.4 图片输入处理流程	5
1.4.1 从本地文件到图像对象	5
1.4.2 检测结果的尺寸映射	6
1.5 表情识别模块分析	6
1.5.1 模型调用链	6
1.5.2 结果排序与概率展示	7
1.5.3 实验现象与局限	7
1.6 年龄与性别估计模块分析	8
1.6.1 模型调用链与输出字段	8
1.6.2 Canvas 标注与结果卡片	8
1.6.3 结果解释与伦理边界	8
1.7 摄像头实时追踪模块分析	9
1.7.1 摄像头授权与视频流绑定	9
1.7.2 周期检测与画布同步	10
1.7.3 结果节流与状态管理	10
1.8 实验复现步骤与观察记录	11
1.8.1 页面访问与资源检查	11
1.8.2 HTML 结构与脚本变量对应	12
1.8.3 建议记录的实验现象	12
1.9 异常场景与边界条件	13
1.9.1 模型文件加载失败	13
1.9.2 输入为空或格式异常	13

1.9.3	摄像头权限与安全上下文	13
1.10	评价指标与实验扩展	14
1.10.1	检测类指标	14
1.10.2	属性估计指标	14
1.10.3	实时性能指标	14
1.11	综合实验结果与分析	15
1.11.1	功能完整性	15
1.11.2	性能影响因素	15
1.11.3	可靠性与数据质量	16
1.12	隐私、安全与改进方向	16
1.13	本章小结	18
参考文献		19

1 基于 face-api.js 的浏览器端人脸分析实验

1.1 实验定位与章节边界

本章分析的是 `code/face-recognition/` 目录中的浏览器端人脸分析实验。该项目以 HTML、CSS 和 JavaScript 静态资源为主体，通过 `face-api.js`[1] 在浏览器中加载预训练模型，并在本地完成图片表情识别、图片年龄与性别估计以及摄像头实时人脸追踪。它的重点不是训练新的神经网络，而是观察一个已经训练好的模型体系如何被前端页面组织、加载、调用和展示。

为了避免与下一章内容混淆，本章只把训练原理作为必要背景来说明：页面中的 `models/` 目录保存的是预训练权重，`public/img/labeled_images` 中的人物图片更接近“参考样本”或“注册样本”，并不会在浏览器端更新神经网络参数。人脸描述子、距离阈值和度量学习的训练机制将在第 2 章结合 `dlib` 示例展开。本章关注的是推理阶段，即输入如何进入模型、模型如何返回结构化结果、页面如何把结果画出来。

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型任务。传统系统常把模型部署在服务器端，客户端只负责采集图像、上传数据和显示结果。这种方式便于集中管理模型，但会带来网络依赖、数据传输、服务部署和延迟控制等问题。随着 `TensorFlow.js` 等前端推理框架的发展，浏览器不仅能显示页面，也能成为模型推理运行环境。对于课程实验而言，这种部署方式具有明显优势：运行入口直观、依赖环境轻、实验过程容易展示，也便于在报告中把“页面结构”和“识别流程”对应起来分析。

本实验选择三个页面作为观察对象：`face_expression_recognition.html` 用于图片表情识别，`age_gender_estimation.html` 用于年龄与性别估计，`webcam_face_tracking.html` 用于摄像头实时追踪。三个页面共享同一个模型目录和前端资源目录，但各自加载不同业务脚本。通过对它们进行对比，可以看出浏览器端人脸分析系统的一般流程：先加载模型，再取得输入，再执行模型推理，最后把检测框、关键点、表情或属性结果渲染到界面上。

本章的实验目标包括四个方面。第一，梳理静态网页项目中页面、样式、脚本、模型权重和示例资源之间的组织关系；第二，分析图片输入与摄像头视频流输入在前端实现上的差异；第三，分别说明表情识别、年龄性别估计和实时追踪三个功能模块的实现逻辑；第四，总结浏览器端推理在性能、稳定性、隐私和实验复现方面的优点与局限。

1.2 项目结构与资源组织

1.2.1 静态项目结构

该项目没有后端服务,也没有复杂的构建入口。页面入口直接位于 `code/face-recognition/` 根目录,业务脚本位于 `dist/`,模型文件位于 `models/`,示例图片、人物注册图片和演示视频位于 `public/`。从实验复现角度看,这种结构的好处是入口清晰:只要浏览器能够访问静态文件,就可以加载页面并触发识别逻辑;对于摄像头页面,还需要浏览器允许页面访问本机摄像头。

项目中的核心库文件是 `dist/face-api.min.js`。它是 `face-api.js` 的浏览器端构建文件,向页面暴露全局对象。三个实验页面并不直接实现底层神经网络,而是调用这个对象提供的高层接口,例如模型加载接口、检测接口、关键点接口、表情接口和绘图接口。业务脚本则负责把这些接口连接成完整实验流程。

页面、脚本和功能之间的对应关系如表 1.1 所示。

表 1.1 浏览器端人脸分析实验页面与脚本对应关系

页面文件	业务脚本	主要功能
<code>face_expression_recognition.html</code>	<code>dist/exprtion_script.js</code>	图片表情识别
<code>age_gender_estimation.html</code>	<code>dist/estimation_script.js</code>	年龄与性别估计
<code>webcam_face_tracking.html</code>	<code>dist/webcam_script.js</code>	摄像头实时追踪

1.2.2 模型权重与本地加载

`models/` 目录中保存了多个预训练模型权重,包括 Tiny Face Detector、SSD MobileNetV1、68 点人脸关键点模型、表情识别模型、年龄性别估计模型和人脸识别模型等。业务脚本中的 均设置为 ,说明当前实验版本从本地目录加载模型,而不是依赖远程 CDN 或 GitHub Raw URL。

本地模型加载对课程报告很重要。首先,它减少了外部网络波动对实验复现的影响。其次,它使报告中的路径说明与实际项目文件保持一致,读者可以直接在仓库中找到模型、脚本和示例资源。最后,本地加载也更容易观察模型文件数量和页面请求过程,有助于理解前端深度学习应用并不是只有一个 HTML 页面,而是由页面、模型权重、脚本和媒体资源共同组成。

需要注意的是,本地保存权重不等于本地训练模型。浏览器页面运行时只通过把权重读入内存,然后执行前向推理。无论用户上传多少图片,模型参数都不会被更新。这个边界决定了本章的分析口径:页面展示的是“如何使用模型”,不是“如何训练模型”。

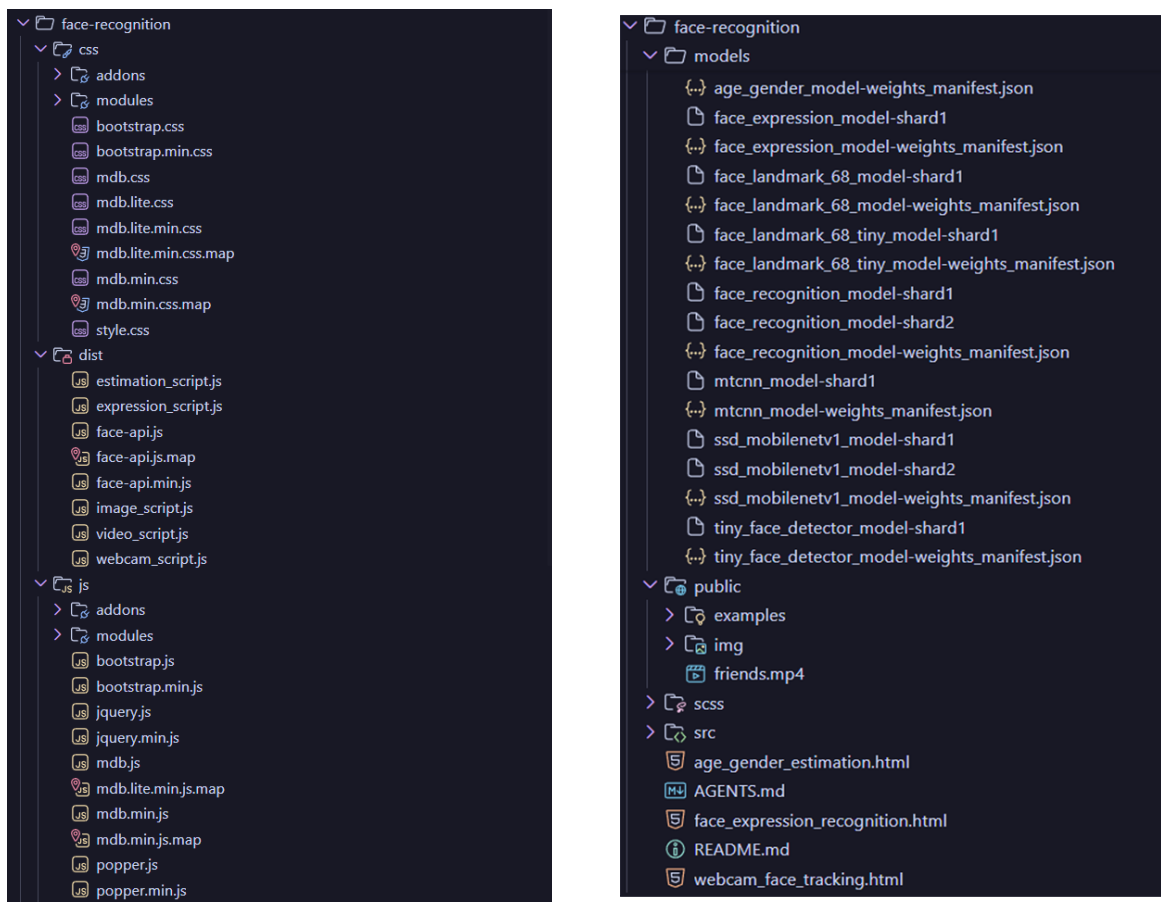


图 1.1 浏览器端人脸分析项目目录结构

1.2.3 页面布局与显示容器

三个页面采用相似的界面组织方式：顶部为导航区域，中间主体为左右分栏。左侧通常放置文件上传控件、摄像头状态、识别结果卡片和空状态提示；右侧放置图片或视频展示区域，并在媒体元素上叠加 Canvas 画布。这样的布局适合实验观察，因为图像标注和结构化结果可以并排出现。用户既能看到检测框是否对准人脸，也能读取模型输出的表情概率、年龄估计、性别置信度等信息。

从职责分工看，HTML 负责定义页面结构，CSS 负责布局和视觉样式，JavaScript 负责模型加载、输入处理、推理调用和结果渲染。页面本身不包含复杂算法，算法能力主要来自 `face-api.js` 和预训练模型。业务脚本则处在连接层：它把 DOM 输入转成模型可处理的媒体对象，把模型输出转成 Canvas 标注和结果卡片。

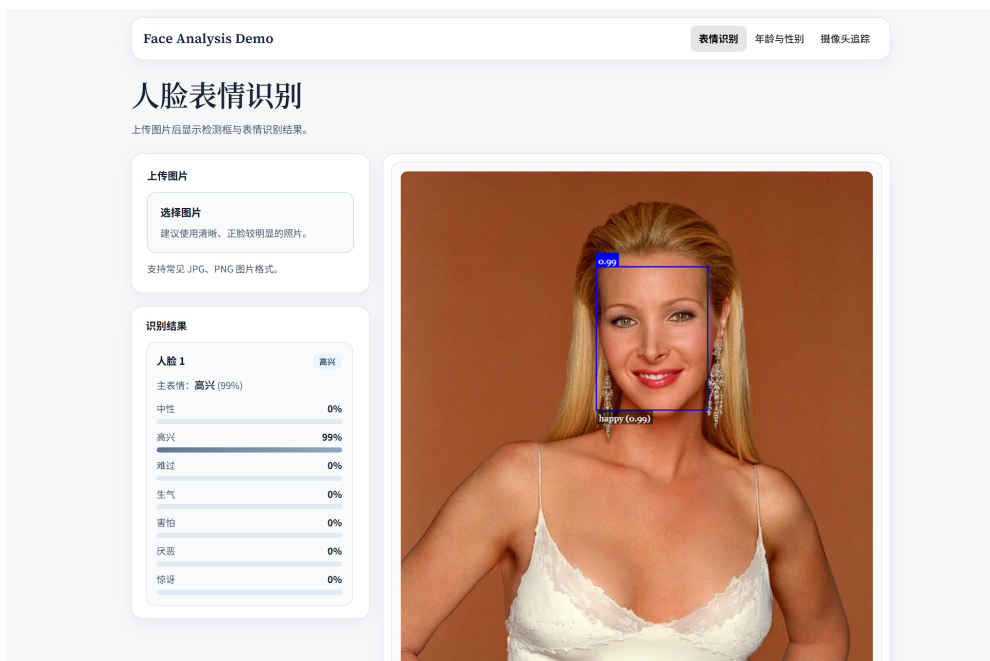


图 1.2 表情识别页面运行截图

1.3 脚本加载与初始化流程

1.3.1 defer 脚本顺序

三个页面都在 `head` 中先引入样式文件，再使用 `defer` 引入脚本。脚本顺序保持为先加载 `dist/face-api.min.js`，再加载当前页面对应的业务脚本。这一顺序是稳定运行的前提，因为业务脚本会直接访问全局对象。如果业务脚本先执行，就会出现之类的错误。

`defer` 的作用是让浏览器在解析 HTML 时并行下载脚本，但等到文档结构解析完成后再按页面中的顺序执行。这样，业务脚本执行时既能访问已经加载好的对象，

也能通过 找到上传控件、结果容器和视频元素。对于静态实验页面来说，这是一种简洁稳定的初始化方式，不需要额外把所有逻辑包在复杂的 DOM ready 回调中。

页面底部还可能引入 jQuery、Popper、Bootstrap 和 MDB 等脚本。这些库主要承担界面组件和样式交互职责，不直接参与模型推理。实验的核心识别链路由 face-api.js 与三个业务脚本完成，其他前端库只改善页面展示体验。

1.3.2 模型加载的并行等待

业务脚本使用 并行加载所需模型。表情识别脚本加载 Tiny Face Detector、68 点关键点、人脸识别、表情识别和 SSD MobileNetV1；年龄性别估计脚本加载 Tiny Face Detector、68 点关键点、人脸识别、年龄性别估计和 SSD MobileNetV1；摄像头脚本加载 Tiny Face Detector、68 点关键点、人脸识别和表情识别模型。只有当这些 Promise 全部完成后，脚本才进入后续的 或 流程。

这种写法体现了前端模型应用的一个基本原则：在用户交互之前先确保模型可用。否则用户可能已经上传图片，但模型权重还没有加载完成，页面就会进入不确定状态。脚本通过等待模型加载完成后再注册上传监听器或请求摄像头，避免了“输入已经到达但模型尚未准备好”的竞态问题。

不同页面加载的模型略有冗余。例如图片页面加载了，但当前展示重点不一定使用身份匹配结果。这种设计可能是为了复用脚本模板或保留扩展空间。就实验报告而言，可以把它理解为典型的人脸分析流水线：检测模型先找到人脸，关键点模型定位面部结构，描述子模型提取特征，属性模型输出表情、年龄或性别。

1.4 图片输入处理流程

1.4.1 从本地文件到图像对象

图片类页面通过 input[type=file] 接收用户上传的 JPG、PNG 等常见图片格式。业务脚本监听上传控件的 事件。如果用户没有选择文件，脚本直接返回；如果存在文件，页面先渲染“正在识别”的空状态提示，然后清除上一张图片 and 上一张画布，避免旧结果与新结果叠在一起。

上传文件并不能直接送入模型。脚本调用，将本地文件转换为浏览器中的图片对象，再把图片插入展示容器。随后，脚本调用 创建覆盖画布，使模型标注能够叠加在图片上。这里的 Canvas 是结果可视化的关键：检测框、表情标签、年龄性别标签都不是写在原图上，而是画在覆盖层中。

脚本根据图片的显示尺寸构造，再调用 使画布尺寸与图片一致。这个步骤容易被忽略，但它直接影响标注位置。如果图片显示尺寸与 Canvas 尺寸不一致，模型

输出框即使在原始坐标中正确，绘制到页面上也可能发生偏移。

1.4.2 检测结果的尺寸映射

模型推理完成后，脚本调用 将检测结果映射到当前显示尺寸。浏览器页面中图片可能受到 CSS 缩放影响，因此模型输出坐标与页面显示坐标不一定完全一致。通过尺寸映射，检测框和属性标签才能准确叠加到用户看到的图片位置上。

图片页面的通用流程可以概括为五步：第一，监听文件上传事件；第二，将文件转换为图片对象并插入页面；第三，创建与图片匹配的 Canvas 覆盖层；第四，调用人脸检测和属性识别接口；第五，将结果映射到显示尺寸，并同时绘制到 Canvas 与渲染到侧边栏。这个流程也说明，浏览器端实验并不是单纯调用一个识别函数，而是包含输入转换、异步等待、尺寸同步、结果排序和界面反馈等多个工程环节。

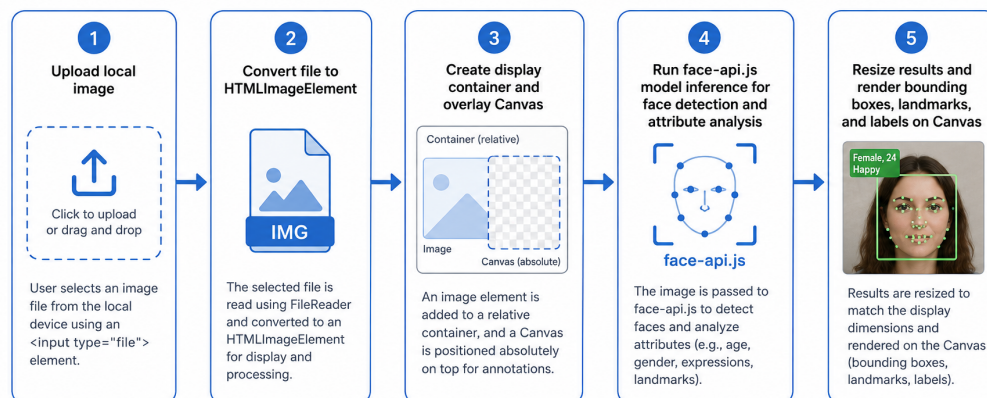


图 1.3 图片输入与画布标注流程

1.5 表情识别模块分析

1.5.1 模型调用链

表情识别由 `dist/expression_script.js` 实现。脚本在图片上传后调用如下链式接口：先执行 检测所有人脸，再串联 获取 68 点关键点，接着调用 提取人脸描述子，最后调用 得到表情分类概率。最终返回的每个人脸结果中包含检测框、关键点、描述子和表情分数。

表情模型输出七类表情概率：、、、、、、。业务脚本将它们映射为中文标签：中性、高兴、难过、生气、害怕、厌恶和惊讶。映射后的标签更适合课程报告和中文界面阅读。

1.5.2 结果排序与概率展示

当图片中存在多张人脸时，脚本并不直接使用模型返回顺序，而是根据检测框的横坐标进行排序，使结果卡片顺序与图片中从左到右的人脸位置一致。这是一个很小但很实用的交互细节。对于多人脸图片，如果侧边栏结果顺序与图像位置不一致，读者很难判断某个概率属于哪张脸。

每张人脸的结果卡片会先显示主表情，即七类概率中分数最高的一项。随后，脚本遍历所有表情类别，用百分比和进度条展示完整概率分布。这样做比只显示一个标签更有解释价值。例如模型可能判断“高兴”为 54%，“中性”为 38%，这说明结果并不是绝对确定，而是在多个类别之间存在竞争。对于课程实验而言，展示概率分布有助于理解模型输出的置信度。

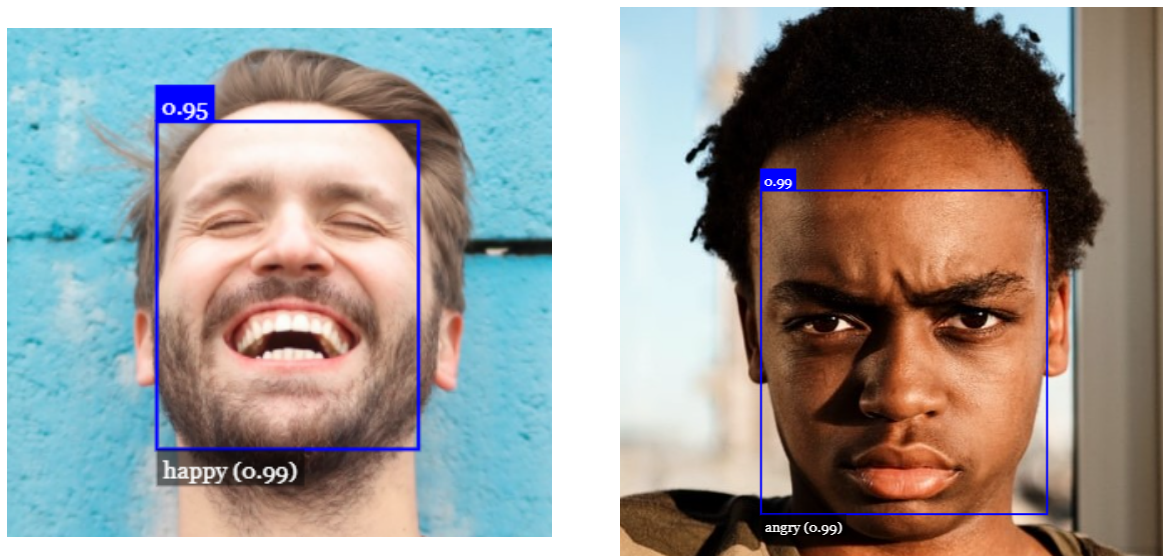


图 1.4 表情识别示例结果

1.5.3 实验现象与局限

表情识别对人脸姿态、遮挡、清晰度和表情幅度比较敏感。正脸、光照均匀、面部无遮挡的图片通常更容易得到稳定结果；侧脸、低分辨率、强背光或面部被手、口罩、头发遮挡时，模型可能无法检测到人脸，或者检测到了人脸但表情概率分布不稳定。表情本身也具有主观性，同一张图片可能同时包含“中性”和“高兴”的特征，因此输出应理解为模型概率估计，而不是人工情绪判断。

从界面反馈看，脚本为“正在识别”和“未检测到人脸”设置了空状态提示。这些状态能降低用户等待时的不确定感，也使页面更接近完整应用，而不是只在控制台打印结果的演示代码。课程报告中记录这些状态，有助于说明实验系统不仅关注模型调用，也关注用户能否理解当前运行阶段。

1.6 年龄与性别估计模块分析

1.6.1 模型调用链与输出字段

年龄与性别估计由 `dist/estimation_script.js` 实现。它与表情识别页面共享类似的图片上传、Canvas 创建和尺寸映射流程，但推理链路末尾换成。模型输出主要包括检测框、估计年龄、性别标签和性别置信度。脚本将 和 映射为“男性”和“女性”，并将年龄四舍五入后以“岁”为单位展示。

该模块体现了多任务人脸属性估计的基本思路。人脸检测模型先确定候选区域，关键点模型提供面部结构信息，描述子模型提取可用于表示身份或面部特征的向量，年龄性别模型再在检测到的人脸区域上输出外观属性。虽然脚本使用链式接口写得很短，但背后实际包含多个模型的顺序协作。

1.6.2 Canvas 标注与结果卡片

年龄性别页面使用 绘制检测框，然后为每张人脸创建，将年龄和性别字符串作为框标签叠加到图像上。同时，侧边栏结果卡片会展示年龄估计、性别判断和性别置信度。图像标注适合快速定位，侧边栏适合读取更完整的结构化结果。

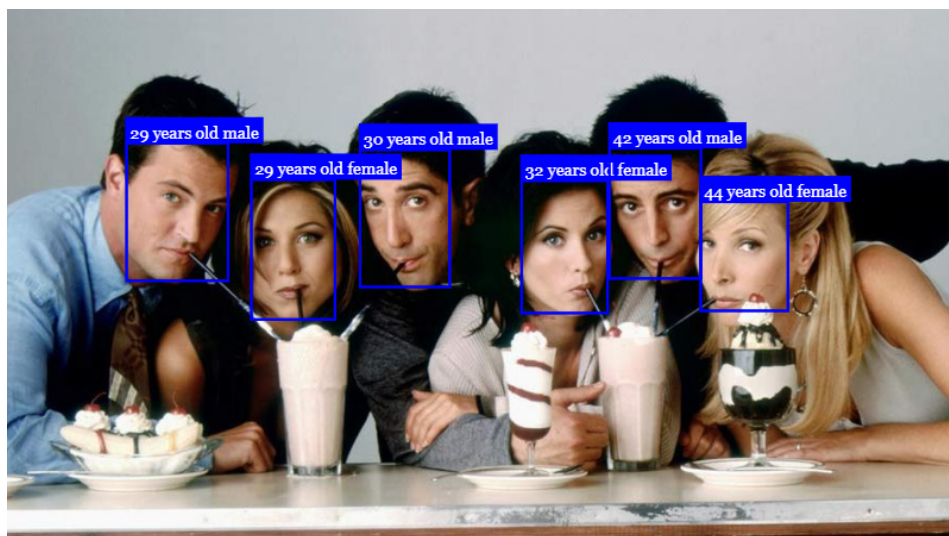


图 1.5 年龄与性别估计示例结果

1.6.3 结果解释与伦理边界

年龄和性别估计结果必须谨慎解释。模型看到的是图像中的外观特征，而不是人的真实身份信息。年龄估计会受到妆容、光照、表情、拍摄角度和图像质量影响；性别估计则涉及更敏感的个人属性，也可能受到训练数据分布和社会偏差影响。因此，本实验中的结果应称为“模型估计”或“外观属性预测”，不应被表述为真实年龄或真实性别。

在实际应用中，这类属性输出不适合直接用于身份、资格、风险、权限等高影响决策。课程实验可以用它展示模型能力和局限，但报告中应明确说明其不确定性。这样的表述既符合技术事实，也能避免把演示模型包装成可靠判断工具。

年龄与性别估计页面的运行过程可以分为输入准备和结果展示两个阶段。图 1.6 展示了用户在页面中选择待分析图片的状态，此时页面已经提供上传入口和图片显示区域，为后续模型推理准备输入数据。

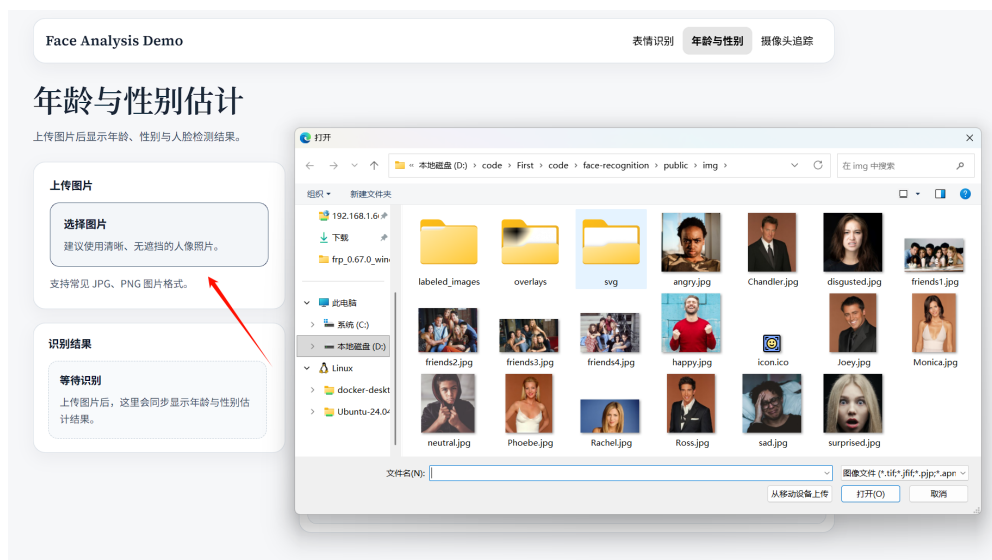


图 1.6 年龄与性别估计页面的图片选择状态

完成图片上传后，脚本调用模型进行人脸检测、年龄估计和性别判断，并将结果同时绘制在图像区域和结果卡片中。图 1.7 中可以看到，页面在检测框附近叠加了年龄与性别标签，侧边栏则以结构化形式展示估计年龄、性别类别和置信度。

1.7 摄像头实时追踪模块分析

1.7.1 摄像头授权与视频流绑定

摄像头实时追踪由 `dist/webcam_script.js` 实现。页面启动后先显示“等待授权”状态，并检查浏览器是否支持媒体采集接口。脚本优先使用 `getUserMedia`，同时保留旧版 `webkitUserMedia` 和 `msUserMedia` 的兼容写法。如果浏览器不支持这些接口，页面会显示“摄像头不可用”；如果用户拒绝授权或设备不可用，页面会显示“访问失败”。

成功获取摄像头视频流后，脚本将 `video` 元素赋给 `videoElement`。此时浏览器负责把摄像头画面播放到页面中的 `video` 元素。只有当视频触发 `loadedmetadata` 事件后，脚本才创建覆盖画布并启动周期性识别。这种顺序很合理，因为视频尺寸和播放状态尚未确定时，提前创建画布容易导致尺寸不匹配。

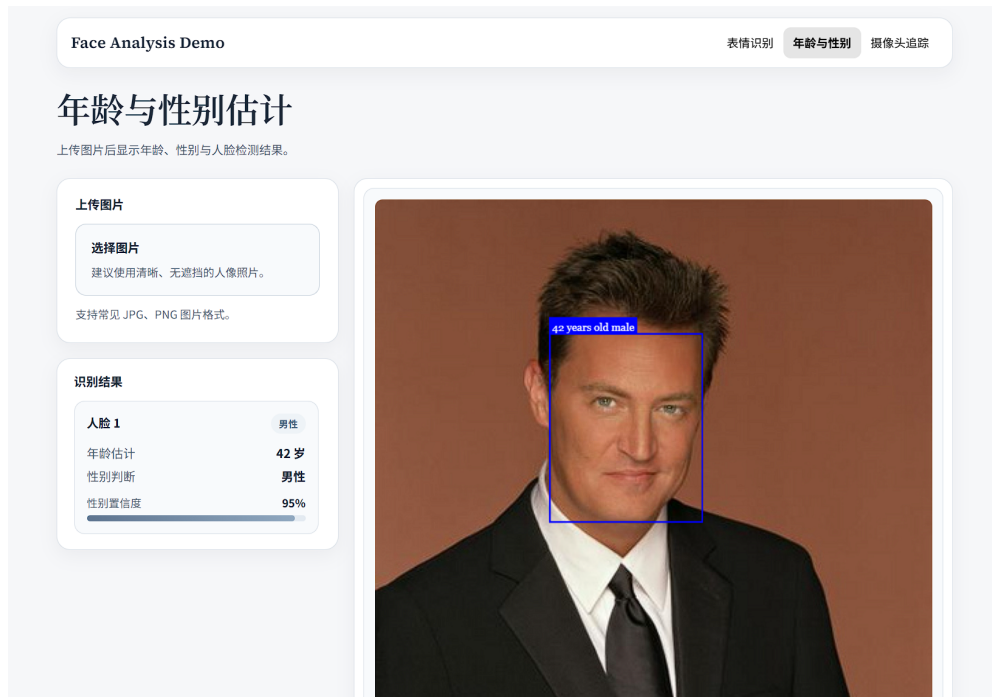


图 1.7 年龄与性别估计页面的结果展示状态

1.7.2 周期检测与画布同步

实时追踪的核心是一个 100 ms 间隔的。每次定时器触发时，脚本读取当前视频帧，调用 检测人脸，再串联关键点定位和表情识别。推理完成后，脚本将结果映射到当前显示尺寸，清空上一帧画布内容，并重新绘制检测框、68 点关键点和表情标签。

与图片页面相比，摄像头页面更强调连续性和性能。图片页面只需要处理一次上传图片，摄像头页面则不断处理视频帧。如果每帧都执行复杂模型，低性能设备可能出现卡顿。因此脚本选择 **Tiny Face Detector** 作为实时检测器，并把识别间隔设置为 100 ms，相当于理论上每秒最多处理约 10 次。这个频率不是视频帧率本身，而是模型推理和标注更新频率。

脚本中的 会根据展示容器尺寸同步 **Canvas** 大小，窗口尺寸变化时也会重新匹配。这个函数解决了实时页面常见的标注错位问题：视频画面可能因为响应式布局变化而改变显示尺寸，如果 **Canvas** 不跟随更新，检测框就会偏离人脸位置。

1.7.3 结果节流与状态管理

摄像头页面不仅要绘制画布，还要更新侧边栏结果。高频更新 **DOM** 会带来额外开销，因此脚本设置了，并用 缓存上一次结果。当本次结果与上次相同，或者距离上次渲染时间不足 280 ms 时，脚本会跳过侧边栏更新。这样可以减少重复 **DOM**

写入，让页面把更多资源留给视频显示和模型推理。

脚本还监听视频的 `ended` 事件，及时清除检测定时器并更新页面状态。这个细节说明实时推理任务需要生命周期管理：当视频已经暂停，如果定时器仍在后台运行，就会浪费计算资源，也可能造成用户误以为识别还在进行。

此处放置摄像头实时追踪截图或 GIF 截帧。
建议文件名：`img/webcam_tracking_runtime.png`。
截图内容建议包含视频画面、检测框、关键点和侧边栏实时结果。

图 1.8 摄像头实时追踪运行效果占位图

1.8 实验复现步骤与观察记录

1.8.1 页面访问与资源检查

复现实验时，首先需要确认项目目录中的静态资源完整。三个 HTML 页面必须能够访问 `css/`、`js/`、`dist/`、`models/` 和 `public/` 等目录。若页面打开后只有布局而没有识别能力，通常需要检查 `dist/face-api.min.js` 是否加载成功；若页面能够显示但长时间无法识别，通常需要检查 `models/` 中的权重文件是否存在，以及浏览器控制台是否出现模型加载失败信息。

对于图片类页面，实验观察可以从三个层次记录。第一层是页面结构是否正常，包括导航栏、上传控件、结果区域和图片展示区域是否出现。第二层是模型初始化是否正常，即上传图片后是否能进入“正在识别”状态并最终绘制结果。第三层是识别结果是否合理，包括检测框是否覆盖人脸、侧边栏结果数量是否与图中人脸数量一致、概率或置信度是否以可读形式展示。

对于摄像头页面，复现步骤还需要额外关注浏览器权限。页面必须在浏览器允许摄像头访问的环境下运行；如果浏览器阻止摄像头权限，页面不会进入实时识别阶段。摄像头页面的观察重点包括状态卡片是否从“等待授权”变为“正在连接”或“摄像头已连接”，视频画面是否正常播放，检测框和关键点是否随人脸移动更新，暂停视频后定时器是否停止。

1.8.2 HTML 结构与脚本变量对应

三个页面的 HTML 结构与脚本变量之间存在明确对应关系。表情识别和年龄性别估计页面都包含 `id="upload"` 的文件输入控件，脚本通过 `imageUpload` 获取它；右侧结果区域使用 `id="analysis-stage"`，脚本把图片和 Canvas 都插入这个区域；侧边栏结果则分别使用 `id="expression-results"` 和 `id="estimation-results"`。

摄像头页面的关键元素不同。视频元素使用 `id="video"`，脚本把摄像头流绑定到该元素；画布覆盖层插入 `id="tracking-stage"` 对应的容器；摄像头状态写入 `id="webcam-status"`；实时识别结果写入 `id="webcam-results"`。这种 DOM 命名方式让页面结构和业务脚本能够直接对应，便于阅读和调试。

从报告写作角度看，记录这些 DOM 对应关系有两个作用。第一，它能证明实验分析来自实际代码，而不是只描述模型概念。第二，它能解释为什么 HTML 中的元素命名会影响脚本运行。如果某个元素的 `id` 被改动，而脚本仍按旧名称查找，页面就会在初始化阶段出现空引用错误。

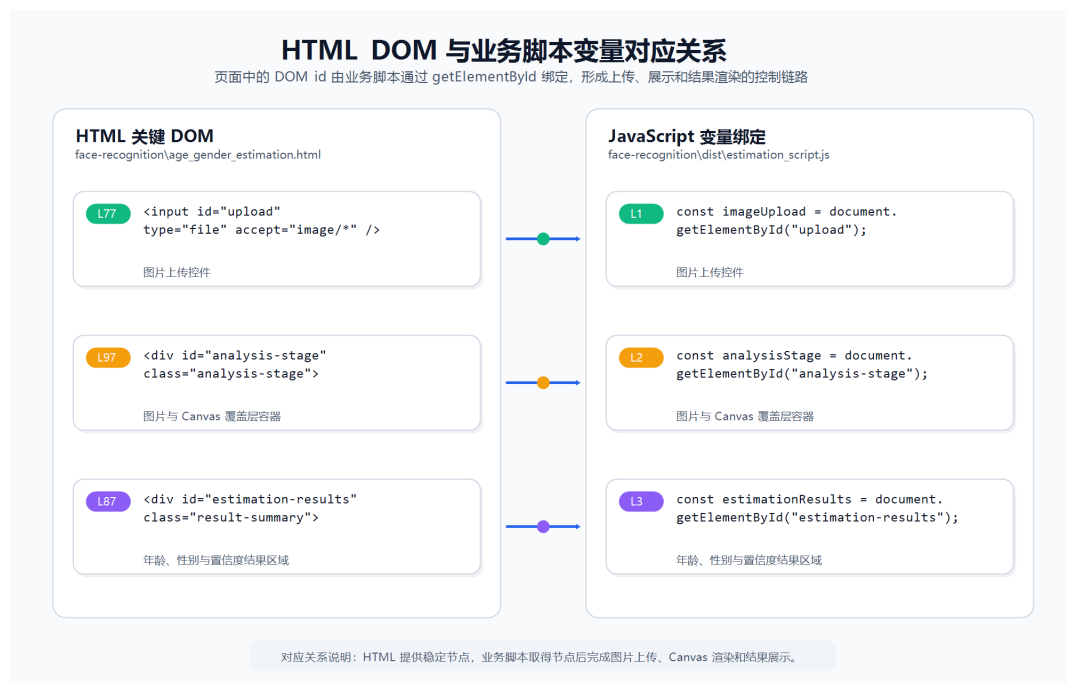


图 1.9 页面 DOM 与业务脚本变量对应关系

1.8.3 建议记录的实验现象

为了让报告更像完整实验，而不是只像代码说明，可以在复现时记录若干具体实验现象。表情识别页面可以分别上传单人正脸、多人合照、侧脸、模糊图片和夸张表情图片，观察检测数量和表情概率变化。年龄性别估计页面可以记录不同年龄段、不同光照和不同姿态图片下的估计波动。摄像头页面可以记录正脸静止、头部

转动、多人进入画面和光照变化时的实时标注稳定性。

这些观察不一定需要形成严格统计实验，但应体现输入条件与输出结果之间的关系。例如，当人脸较小或模糊时，检测框可能消失；当表情不明显时，中性概率可能占主导；当摄像头画面快速移动时，关键点可能出现短暂抖动。把这些现象写入报告，可以避免只呈现成功案例，也能体现对模型局限的理解。

1.9 异常场景与边界条件

1.9.1 模型文件加载失败

浏览器端实验最常见的失败场景之一是模型文件加载失败。如果 `MODEL_URL` 指向错误目录，或者 `models/` 中缺少权重文件，将无法进入后续流程。用户看到的现象可能是页面可以打开，但上传图片后没有反应，或者摄像头页面一直停留在初始状态。此时应优先检查浏览器控制台和网络请求列表。

由于模型加载是异步过程，页面最好提供加载失败提示。当前脚本主要处理正常加载路径，对于模型加载异常的用户反馈仍有改进空间。课程报告可以把这一点列为后续优化方向：在 后补充 分支，提示“模型资源加载失败，请检查 `models` 目录或网络路径”。

1.9.2 输入为空或格式异常

图片页面已经对空文件选择做了简单保护：当 为 0 时，脚本直接返回。由于上传控件使用 `accept="image/*"`，浏览器会尽量限制用户选择非图片文件。但这种限制并不是严格安全边界，实际应用仍应在脚本中处理图片解码失败的异常。

如果图片过大，浏览器端推理可能明显变慢，甚至造成页面短暂无响应。报告中可以说明，课程实验适合使用清晰但尺寸适中的图片。实际应用中可以在前端对图片进行压缩、缩放或限制最大尺寸，再送入模型推理，从而减少计算开销。

1.9.3 摄像头权限与安全上下文

摄像头页面还可能受到浏览器安全策略影响。现代浏览器通常要求摄像头访问发生在安全上下文中，例如 `HTTPS` 或本机 `localhost` 环境。若直接以某些文件协议打开页面，摄像头接口可能不可用。脚本虽然兼容新旧 接口，但无法绕过浏览器权限策略。

用户拒绝授权、设备被其他程序占用、摄像头不存在或浏览器不支持媒体采集接口，都会导致实时追踪失败。当前脚本会显示“访问失败”或“不支持”等状态，这对于实验页面已经比较友好。后续若扩展为正式应用，还可以增加更具体的错误

分类，例如权限拒绝、设备未找到、设备忙碌和安全上下文不满足。

1.10 评价指标与实验扩展

1.10.1 检测类指标

如果需要把本实验从功能展示扩展为定量分析，可以首先设计检测类指标。对于人脸检测，可统计测试图片总数、真实人脸总数、成功检测人脸数、漏检数和误检数。检测成功率可以粗略表示为成功检测人脸数与真实人脸总数之比。对于多人脸图片，还可以记录检测框位置是否与真实人脸区域基本重合。

课程报告不一定需要构建大规模数据集，但可以选取若干代表性样例：正脸单人、多人合照、侧脸、强光、弱光、遮挡、低清晰度等。通过这些样例记录模型表现，可以让实验结论更有依据。例如，报告可以说明“在正脸清晰图片上识别较稳定，在遮挡和侧脸场景中容易漏检”。

1.10.2 属性估计指标

对于表情识别，评价可以观察主表情是否符合人工直观判断，以及概率分布是否集中。对于年龄估计，可以记录模型估计值与人工标注年龄的大致偏差，但要注意隐私和数据来源。对于性别估计，不建议把结果作为强标签评价，更适合讨论模型输出的不确定性和潜在偏差。

属性估计的结果不应只追求“准确率”一个数字。表情、年龄和性别都涉及主观性、社会属性和数据偏差。更合理的写法是结合置信度、输入质量和适用边界进行分析。尤其在课程报告中，应把这些输出理解为模型能力展示，而不是可靠身份判断。

1.10.3 实时性能指标

摄像头页面可以记录实时性能指标，例如模型更新间隔、页面体感延迟、检测框抖动情况和设备资源占用。当前脚本的检测间隔设置为 100 ms，但实际推理耗时可能超过 100 ms；当推理时间较长时，页面无法真正达到每秒 10 次稳定更新。报告中可以说明，定时器间隔是目标频率，实际频率还取决于设备性能和模型耗时。

如果后续要优化实时性能，可以考虑降低视频显示分辨率、减少绘制内容、只绘制检测框不绘制关键点、延长检测间隔，或者使用更轻量的检测参数。优化方向需要在速度、稳定性和结果丰富度之间取舍。

1.11 综合实验结果与分析

1.11.1 功能完整性

从功能完整性看，该项目完成了三个典型的人脸分析任务。表情识别页面能够在上传图片后检测多张人脸，并输出每张人脸的主表情和七类表情概率；年龄与性别估计页面能够输出检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在获得权限后持续处理视频帧，并实时叠加检测框、关键点和表情结果。

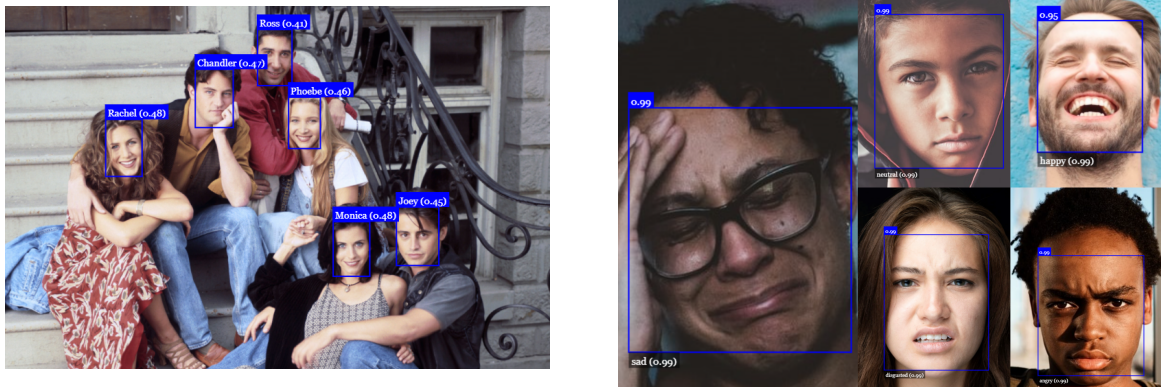


图 1.10 人脸检测与综合演示效果

从工程实现看，项目具有较好的演示性。页面入口清楚，模型资源保存在本地，业务脚本集中在 `dist/`，示例图片和人物图片保存在 `public/`。这种组织方式方便教师讲解，也方便学生对照代码与报告复现实验。相比只展示最终效果，报告中分析文件结构和脚本流程更能体现课程实验的完整性。

1.11.2 性能影响因素

浏览器端推理的性能主要受三个因素影响。第一是模型加载时间。首次打开页面时，浏览器需要读取多个模型权重文件，模型越多，初始等待越明显。第二是单次推理时间。图片越大、人脸越多、模型越复杂，推理时间越长。第三是页面渲染成本。Canvas 绘制和 DOM 更新虽然不属于神经网络计算，但在实时任务中也会影响流畅度。

摄像头页面还受到设备摄像头分辨率、CPU/GPU 能力、浏览器实现和当前系统负载影响。脚本通过 Tiny Face Detector、100 ms 检测间隔和侧边栏节流来控制开销，但并不能保证所有设备都有相同效果。如果输入视频分辨率较高或设备性能较弱，实时标注可能出现延迟、掉帧或结果抖动。

1.11.3 可靠性与数据质量

人脸分析结果对输入质量依赖明显。正脸、光照均匀、面部无遮挡的图像通常更容易得到稳定结果；侧脸、模糊、强背光、表情幅度小、多人人脸重叠或遮挡严重的图像可能导致漏检、误检或属性估计偏差。对于摄像头场景，用户移动、光照变化和镜头自动曝光也会影响每一帧结果。

因此，实验报告不应只写“系统能够识别”，还应写清它在什么条件下更容易识别、在什么条件下可能失败。这样的分析能体现对模型应用边界的理解。尤其是年龄、性别和表情等属性识别，输出本身带有概率性和不确定性，不适合脱离实验语境做确定判断。

1.12 隐私、安全与改进方向

摄像头功能涉及人脸和视频流等敏感信息。当前项目在浏览器端本地处理数据，没有主动把视频流上传到服务器，这是浏览器端部署的一个隐私优势。但是，只要页面请求摄像头权限，就应该让用户理解授权含义。课程报告中应说明：实验只用于本地演示，不应保存、传播或公开未经同意的人脸图像。

从工程改进角度看，项目还可以继续优化。第一，可以抽取公共逻辑，例如模型加载、空状态渲染、检测结果排序、Canvas 尺寸同步等，减少三个脚本之间的重复。第二，可以加入更明确的加载进度和错误提示，例如模型文件缺失、图片格式不支持、摄像头权限拒绝等。第三，可以构造定量测试集，统计不同光照、姿态、遮挡和多人脸场景下的检测成功率、属性稳定性和实时帧率。

从实验写作角度看，后续还可以补充更多真实运行截图。图片页面截图能证明上传和标注流程正常，摄像头截图能证明实时追踪有效，目录结构截图能证明模型和资源均来自本地仓库。相比只放模型示例图，真实运行截图更能体现课程项目的复现过程。

表情识别页面如图 1.11 所示。该页面以图片上传为入口，右侧用于显示待分析图片和 Canvas 标注结果，左侧则展示表情类别和概率分布，适合观察单张图片中多个人脸的表情识别效果。

年龄与性别估计页面如图 1.12 所示。它与表情识别页面采用相似的上传和展示结构，但侧边栏结果重点变为估计年龄、性别类别和置信度，体现了同一套前端推理流程在不同属性任务中的复用。

摄像头追踪页面如图 1.13 所示。该页面以实时视频流作为输入，在授权摄像头后持续更新检测框、关键点和表情结果，更能体现浏览器端模型推理在连续输入场



图 1.11 表情识别页面运行界面



图 1.12 年龄与性别估计页面运行界面

景下的交互特点。

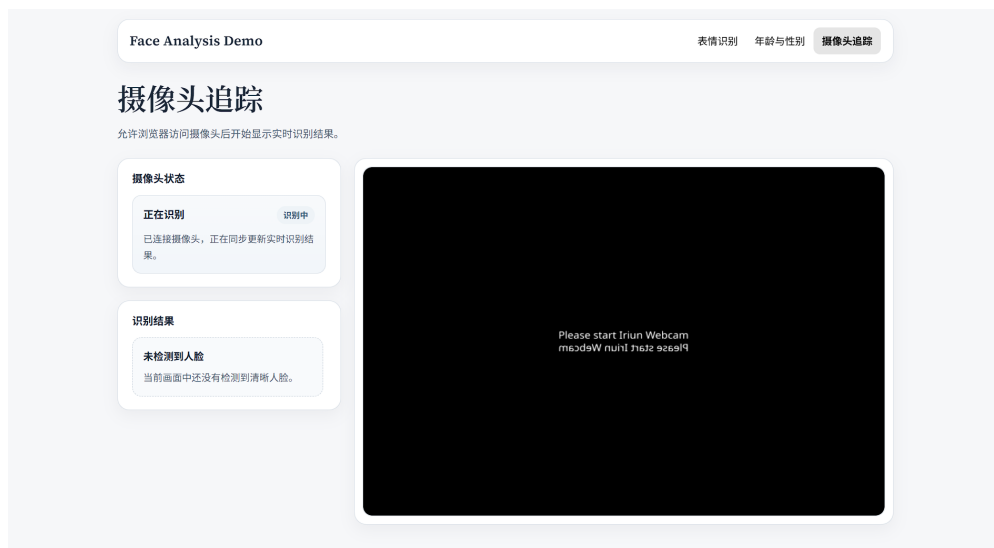


图 1.13 摄像头实时追踪页面运行界面

1.13 本章小结

本章围绕 `face-recognition` 项目分析了浏览器端人脸分析实验。实验展示了静态网页如何利用 `face-api.js` 加载本地预训练模型，并在图片和摄像头视频流两类输入上完成人脸检测、关键点定位、表情识别以及年龄性别估计。章节重点放在前端推理阶段，包括页面结构、模型加载、输入转换、Canvas 绘制、结果渲染和状态管理。

通过三个模块的对比可以看出，浏览器端深度学习推理适合课程演示、交互原型和轻量级体验场景。它降低了后端部署成本，也减少了敏感图像上传的需求。但它仍然受到设备性能、浏览器环境、输入质量和模型偏差影响。下一章将从 `dlib` 开源示例出发，进一步分析人脸描述子模型在训练阶段如何通过深度度量学习获得可用于距离匹配的特征空间。

参考文献

- [1] Vincent Muhler. face-api.js: Javascript api for face detection and face recognition in the browser and node.js with tensorflow.js. GitHub repository, 2018.
- [2] Vincent Muhler. face-api.js model weights. GitHub repository.
- [3] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 2009.
- [4] Davis E. King. dlib c++ library. GitHub repository.
- [5] World Wide Web Consortium. Media capture and streams. W3C Recommendation.
- [6] Bootstrap Team. Bootstrap documentation.